

Name	Janet Oluwatosin Michael
Course	Malware Analysis
Legacy Course Code	CIP-A106: TeslaCrypt Ransomware Unpacking and Memory Analysis
Student ID	2026/EHIT/17073
Cohort	Cohort 11
Assessment Submission	05 October 2026 23:59 WAT
Instructor	Aminu Idris

Executive Summary

In this lab, I analysed a TeslaCrypt ransomware sample in a Windows virtual machine. I used tools such as PEStudio, x32dbg, System Informer, PowerShell and WinPmem to carry out the analysis.

The sample was detected by Windows Defender as Ransom:Win32/TeslaCrypt.GNM!MTB. The Defender result also showed DidThreatExecute: False, meaning that the malware was detected but was not confirmed to have executed.

From the PEStudio analysis, I found that the file was a 32-bit executable with a size of 368,640 bytes. The SHA-256 hash of the sample was 5343947829609F69E84FE7E8172C38EE018EDE3C9898D4895275F596AC54320D. I also noticed that some of the sections had high entropy and the section layout looked unusual. This made me think that the file could be packed or obfuscated, but I could not confirm the exact packer.

I used x32dbg to look at the entry point of the sample. The entry point was 0x00403C40, and I observed a jump to 0x004013D0. I was not able to fully confirm the original entry point or obtain a validated unpacked copy.

I also used System Informer to identify the malware process. The process ID was 7068 and its parent process was x32dbg with PID 7208. I then used WinPmem to acquire the memory of the system. The memory file was named TeslaCrypt_memory.raw and its SHA-256 hash was B9463ADBD4C6B5A6A21AFEB87C5DD903EA61459F07BE34535DDB820BB5000467.

The analysis was done in an isolated environment with the network disabled. I did not observe confirmed ransomware activities such as file encryption or communication with a command-and-

control server. Some parts of the analysis, especially confirming the unpacked code and validating the memory contents, were not fully completed.

2. Introduction

The aim of this lab was to analyse a TeslaCrypt ransomware sample in a controlled and isolated Windows virtual machine. The main things I focused on were checking the sample with PEStudio, looking at its entry point using x32dbg, checking the running process with System Informer, and acquiring a memory image using WinPmem.

I also checked the file hash and other information about the sample so that the file could be identified and the evidence could be recorded. The analysis was done without an active network connection after the required files and tools had been downloaded.

3. Environment and Methodology

At first, I planned to use FLARE-VM for the malware analysis, but I was unable to download it successfully. Because of this, I decided to use the Windows virtual machine that was already available to me and install the required malware analysis tools separately. I created a clean snapshot called CIP-A106_Lab03_Clean_Baseline before working with the sample. I used NAT temporarily to download the required tools and the TeslaCrypt sample. After downloading them, I disabled the network connection and continued the analysis in the isolated virtual machine.

The main tools I used were PEStudio, x32dbg, System Informer, PowerShell and WinPmem. PEStudio was used for the initial static analysis, x32dbg was used to examine the entry point, System Informer was used to check the malware process, PowerShell was used for system and Defender information, and WinPmem was used to acquire the memory image.

The TeslaCrypt sample was kept inside the virtual machine during the analysis. I did not upload the sample or the memory dump to any public website.

4. Sample Information

The sample I analysed was demo1_ransomware.bin. It was obtained from the TeslaCrypt ransomware lab sample provided for the exercise.

The file was **368,640 bytes** in size and was identified as a **32-bit Windows executable**. PEStudio showed the following SHA-256 hash:

5343947829609F69E84FE7E8172C38EE018EDE3C9898D4895275F596AC54320D

Windows Defender detected the file as: Ransom:Win32/TeslaCrypt.GNM!MTB

The Defender result also showed: DidThreatExecute : False

This means the file was detected as a TeslaCrypt ransomware sample, but there was no evidence from Defender that the detected threat had executed.

The original TeslaCrypt archive also had the following SHA-256 hash:

63E52C4A52CBFB9AB1885CBB83E5D5B836802A06618B3BC07EBC20545DF75729

I recorded these hashes so that the files used during the analysis could be identified and checked against the evidence collected.

5. Initial Static Analysis

I used PEStudio to carry out the initial static analysis of the TeslaCrypt sample. The file was identified as a 32-bit executable with a GUI type and a size of 368,640 bytes.

PEStudio showed that the sample was compiled with **Microsoft Linker 8.0 / Visual Studio 2005**. The compile timestamp shown was **28 February 2016, 18:15:11**. The file description was nah nah and the version shown was 1.600.5512.

The entry point was shown as:

0x00003C40

The entry point was located in the .text section.

The sample contained several sections, including:

- .text
- para
- .rdata
- .data
- .crt
- CODE
- .erloc
- .rsrc

I also noticed that some of the sections had high entropy. For example, .data had an entropy of **7.648**, .crt had **7.883**, and CODE had **7.860**. Because of this, I considered that the file could have been packed or obfuscated. However, I could not confirm a specific packer from PEStudio alone.

Some of the imports I observed included GlobalMemoryStatus, CreateEventW, GetClusterResourceKey, RemovePropA, memcpy and memset.

PEStudio also showed a server-related indicator saying:

The server name or address could not be resolved

I recorded this as an observation from the tool and did not treat it as a confirmed network IOC because I did not identify a specific domain or IP address from the sample.

6. Packer Assessment

From the PESTudio results, I could not confirm that the sample was packed with a specific packer. However, I noticed that some of the sections had high entropy and the section layout was unusual.

The highest entropy I observed was in the .crt section with **7.883**, followed by CODE with **7.860** and .data with **7.648**. These values made me suspect that the file may contain compressed or obfuscated data.

I did not use the high entropy alone to say that the file was packed because high entropy does not always mean that a file is packed. I therefore treated packing as a possibility that needed to be checked further using the debugger.

7. Debugger Analysis

I used **x32dbg** to examine the TeslaCrypt sample because the file was a 32-bit executable. I loaded demo1_ransomware.bin into the debugger and checked the entry point shown by PESTudio.

The entry point was:

0x00403C40

At this address, I observed the following instructions:

00403C40 8935 A04D4100 mov dword ptr ds:[414DA0],esi

00403C46 55 push ebp

00403C47 54 push esp

00403C48 893D A44D4100 mov dword ptr ds:[414DA4],edi

00403C4E 8F05 B04D4100 pop dword ptr ds:[414DB0]

00403C54 891D A84D4100 mov dword ptr ds:[414DA8],ebx

00403C5A 8F05 AC4D4100 pop dword ptr ds:[414DAC]

00403C60 8935 A04D4100 mov dword ptr ds:[414DA0],esi

00403C66 E9 65D7FFFF jmp 004013D0

The instruction at 0x00403C66 showed a jump to 0x004013D0. I also recorded the register values at the entry point, including EIP = 00403C40 and ECX = 00403C40.

I tried to follow the execution further in x32dbg, but I was not able to conclusively identify the original entry point or confirm that 0x004013D0 was the unpacked code. Because of this, I did not treat 0x004013D0 as the confirmed OEP.

The debugger analysis therefore confirmed the initial entry point and the observed jump, but the unpacking process could not be fully confirmed.

8. Process Analysis

I used **System Informer** to check the processes running during the analysis. The TeslaCrypt sample was running as:

- **Process:** demo1_ransomware.bin
- **PID:** 7068
- **Parent process:** x32dbg.exe
- **Parent PID:** 7208

The sample was being run through x32dbg during the analysis, which explains why x32dbg appeared as its parent process.

I also observed other normal Windows processes running in the system, such as explorer.exe, csrss.exe, winlogon.exe, dwm.exe and powershell.exe.

I used the process information mainly to identify the TeslaCrypt process and record its PID before acquiring the memory image. I did not observe confirmed ransomware activity such as file encryption or other changes to the system during the analysis.

9. Memory Acquisition

After identifying the TeslaCrypt process, I used **WinPmem** to acquire a memory image from the Windows virtual machine. The TeslaCrypt process had PID **7068** and was running under x32dbg.

The WinPmem command I used was:

```
.\go-winpmem_amd64_1.0-rc2_signed.exe acquire TeslaCrypt_memory.raw
```

The acquisition completed successfully. WinPmem created the following memory artifact:

- **Filename:** TeslaCrypt_memory.raw
- **Tool:** WinPmem
- **Acquisition time:** 5 October 2026, 1:54:02 AM
- **Size:** 4,777,312,256 bytes

- **SHA-256:**
B9463ADBD4C6B5A6A21AFEB87C5DD903EA61459F07BE34535DDB820BB5000467

The WinPmem output showed that the memory acquisition was completed in about 2 minutes. The memory file is a **full physical memory image**, rather than a memory dump containing only the TeslaCrypt process. I recorded this as a limitation because the assignment asks for relevant memory evidence, but the tool acquired the full system memory.

10. Memory Artifact Validation

After acquiring the memory image, I recorded its filename, size and SHA-256 hash for identification. The acquired file was TeslaCrypt_memory.raw and its SHA-256 hash was:

B9463ADBD4C6B5A6A21AFEB87C5DD903EA61459F07BE34535DDB820BB5000467

The memory image was acquired while the TeslaCrypt process was running under x32dbg. However, I was not able to complete a full validation of the memory contents, such as identifying specific unpacked code, memory regions or modules belonging to the sample.

Because of this, I did not make any claim that the memory image contained confirmed unpacked TeslaCrypt code. The hash and acquisition details were recorded as evidence.

11. Indicators of Compromise (IOCs)

During my analysis, I recorded the indicators that were directly shown by the tools I used. I did not add indicators that I could not confirm from the sample.

Indicator	Value	Source
File name	demo1_ransomware.bin	PEStudio / Windows
SHA-256	5343947829609F69E84FE7E8172C38EE018 EDE3C9898D4895275F596AC54320D	PEStudio
File size	368,640 bytes	PEStudio
Defender detection	Ransom:Win32/TeslaCrypt.GNM!MTB	Windows Defender
Process	demo1_ransomware.bin	System Informer
Process ID	7068	System Informer
Parent process	x32dbg.exe	System Informer
Parent PID	7208	System Informer
Memory artifact	TeslaCrypt_memory.raw	WinPmem
Memory SHA-256	B9463ADBD4C6B5A6A21AFEB87C5DD903EA6 1459F07BE34535DDB820BB5000467	WinPmem

PEStudio also showed the message “**The server name or address could not be resolved.**” I recorded this as an observation, but I did not treat it as a confirmed network IOC because no specific domain or IP address was identified.

12. Ransomware Behaviour

During the analysis, I did not observe confirmed ransomware behaviour such as file encryption, changing file extensions, deleting files, creating persistence or communicating with a command-and-control server.

Windows Defender detected the sample as Ransom:Win32/TeslaCrypt.GNM!MTB, but the Defender result showed DidThreatExecute : False. Therefore, I cannot say from my evidence that the ransomware actually carried out encryption or other malicious actions during my analysis.

13. MITRE ATT&CK Mapping

Based on the evidence I collected, I could not confidently map many MITRE ATT&CK techniques because I did not observe the ransomware carrying out specific actions during the analysis.

One possible technique is:

MITRE ATT&CK Technique	Evidence	Assessment	Confidence
T1027 – Obfuscated/Compressed Files or Information	High entropy was observed in some sections, including .crt, CODE and .data, and the section layout was unusual.	The file may contain compressed or obfuscated information. However, I could not confirm a specific packer.	Medium

I treated this as a **possible technique** rather than confirmed malicious behaviour because high entropy by itself does not prove that the file uses packing or obfuscation.

14. Confidence Levels and Limitations

I had high confidence in the basic information collected from the sample, including the file size, SHA-256 hash, file type and Defender detection. The process information from System Informer and the memory acquisition details from WinPmem were also directly observed and recorded.

I had medium confidence in the possibility that the sample was packed or obfuscated because some sections had high entropy and the section layout was unusual. However, I could not confirm the exact packer. The main limitation was the debugger analysis. I was able to identify

the entry point at 0x00403C40 and observe a jump to 0x004013D0, but I could not confirm that this address was the original entry point or prove that the sample had been completely unpacked.

Another limitation was the memory analysis. I successfully acquired the physical memory image and calculated its hash, but I did not complete a detailed analysis of the memory regions and modules to confirm the presence of unpacked TeslaCrypt code. Because of these limitations, I only included findings that were supported by the evidence I collected.

15. Conclusion

In this lab, I was able to carry out the main stages of the TeslaCrypt sample analysis in an isolated Windows virtual machine. I used PESTudio to examine the file, x32dbg to look at the entry point, System Informer to check the running process, and WinPmem to acquire the memory image.

The static analysis showed some unusual sections and high entropy, which made packing or obfuscation possible, but I could not confirm the exact packer. I also confirmed the entry point at 0x00403C40 and observed a jump to 0x004013D0, but I could not confirm the OEP or obtain a validated unpacked copy.

The memory acquisition was successful and the memory artifact was hashed for evidence. However, I did not complete a detailed validation of the memory contents.

Overall, the analysis helped me understand how to examine a ransomware sample safely and how to record evidence such as hashes, process information, debugger observations and memory acquisition details. I also learned that I should not assume that something is malicious behaviour unless I have evidence to support it.

16. Lessons Learned

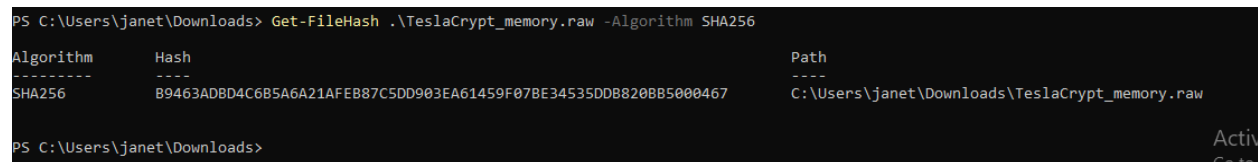
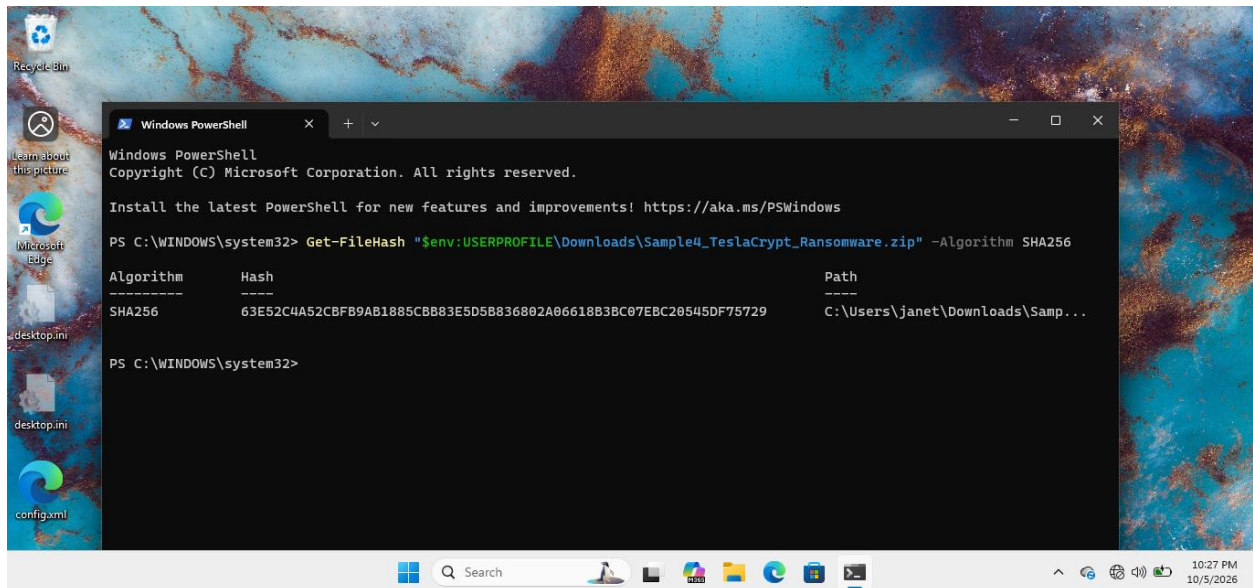
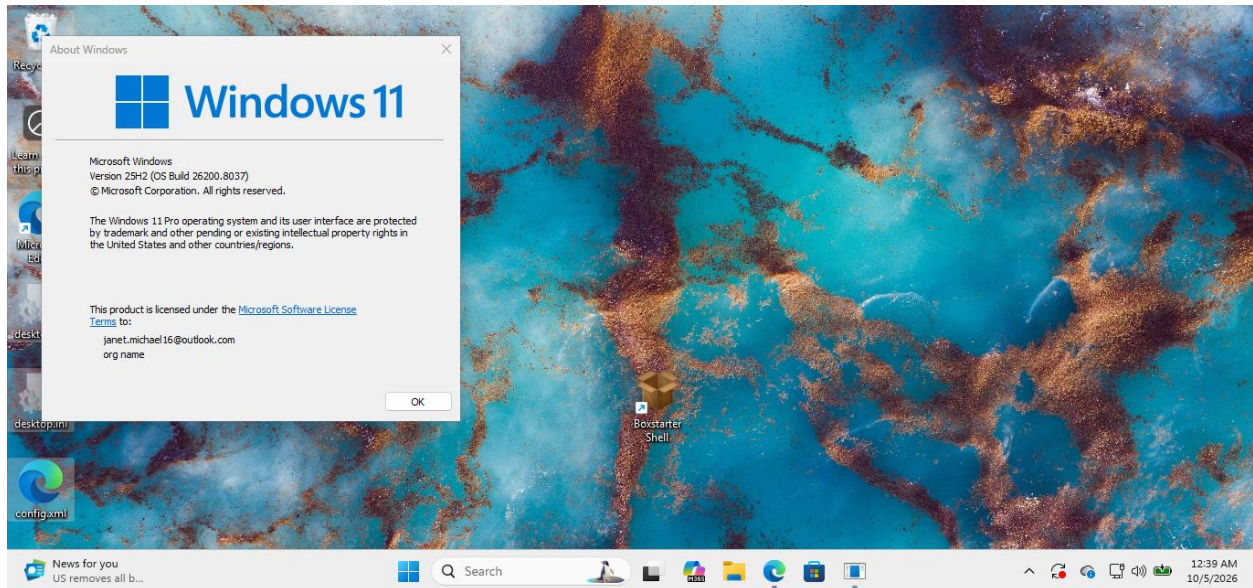
During this lab, I learned how important it is to keep malware analysis in an isolated environment. I also learned how to use different tools for different parts of the analysis instead of depending on one tool.

I learned how PESTudio can help with checking the basic information of a file, its sections, imports and entropy. I also learned how to use x32dbg to locate and examine an executable's entry point.

Another thing I learned was how to identify the malware process using System Informer and acquire a memory image using WinPmem. I also learned that having a hash for files and evidence is important because it helps to confirm exactly what was analysed.

One challenge I had was not being able to use FLARE-VM because I could not download and install it, so I used the Windows VM and installed the tools I needed separately. I also found it difficult to confirm the OEP and fully validate the memory contents. This showed me that I should only report what I can actually support with my evidence.

17. Screenshots



pestudio 9.61 - Malware Initial Assessment - www.wintor.com | c:\users\janet\downloads\teslacrypt_sample\demo1_ransomware.bin (read-only)

file settings about

c:\users\janet\downloads\teslacrypt_sample\demo1_ransomware.bin

property value

file	
file > sha256	5343947829609F69E84FE7E8172C38EE018EDE3C9898D4895275F596AC54320D
file > first 32 bytes (hex)	4D 5A 90 00 03 00 00 00 04 00 00 00 FF 00 00 B8 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
file > first 32 bytes (text)	MZ.....@.....
file > info	size: 368640 bytes, entropy: 7.629
file > type	executable, 32-bit, GUI
file > version	1.600.5512
file > description	nah nahApp
entry-point > first 32 bytes (hex)	89 35 A0 4D 41 00 55 54 89 3D A4 4D 41 00 8F 05 B0 4D 41 00 89 1D A8 4D 41 00 8F 05 AC 4D 41 00
entry-point > location	0x00003C40 (section[.text])
file > signature	Microsoft Linker 8.0
stamps	
stamp > compiler	Sun Feb 28 18:15:11 2016 (UTC)
stamp > debug	Sun Feb 28 18:15:11 2016 (UTC)
stamp > resource	n/a
stamp > import	n/a
stamp > export	n/a
names	
file > name	c:\users\janet\downloads\teslacrypt_sample\demo1_ransomware.bin
debug > file	E:\Tools\ao\fed\release\osc.pdb
export	n/a
version > original-file-name	nah nah
manifest	n/a

sha256 > 5343947829609F69E84FE7E8172C38EE018EDE3C9898D4895275F596AC54320D cpu > 32-bit file > type > executable subsystem > GUI entry-point > 0x00003C40

pestudio 9.61 - Malware Initial Assessment - www.wintor.com | c:\users\janet\downloads\teslacrypt_sample\demo1_ransomware.bin (read-only)

file settings about

c:\users\janet\downloads\teslacrypt_sample\demo1_ransomware.bin

imports (6)	flag (1)	type	ordinal	first-thunk (IAT)	first-thunk-original (INT)	library
GlobalMemoryStatus	x	implicit	-	0x00005260	0x00005260	KERNEL32.dll
CreateEventW	-	implicit	-	0x00005250	0x00005250	KERNEL32.dll
GetClusterResourceKey	-	implicit	-	0x0000520C	0x0000520C	CLUSAPI.dll
RemovePropA	-	implicit	-	0x00005284	0x00005284	USER32.dll
memcpy	-	implicit	-	0x0000523A	0x0000523A	msvcrt.dll
memset	-	implicit	-	0x00005230	0x00005230	msvcrt.dll

sha256 > 5343947829609F69E84FE7E8172C38EE018EDE3C9898D4895275F596AC54320D cpu > 32-bit file > type > executable subsystem > GUI entry-point > 0x00003C40

demo1_ransomware.bin - PID: 7068 - Module: demo1_ransomware.bin - Thread: Main Thread 4908 - x32dbg

File View Debug Tracing Plugins Favourites Options Help May 27 2026 (TitanEngine)

Check for Updates

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads Handles Trace

EAX 001AFFCC
EBX 002D2000
ECX 00403C40 <demo1_ransomwar
EDX 00403C40 <demo1_ransomwar
EBP 001AFFB4 "[1]7u"
ESP 001AFF78 <demo1_ransomwar
ESI 00403C40 <demo1_ransomwar
EDI 00403C40 <demo1_ransomwar
EIP 00403C40 <demo1_ransomwar
EFLAGS 00010346
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 1 IF 1
LastError: 00000000 (ERROR: FNVVAR ^
Default (stdcall) 5 Calls

1: [esp+4] 002D2000 002D2000
2: [esp+8] 753F5D30 <kernel32.BaseT
3: [esp+C] 001AFFDC 001AFFDC
4: [esp+10] 774FD818 ntdll.774FD818
5: [esp+14] 002D2000 002D2000

demo1_ransomware.00401300

.text:00403C66 demo1_ransomware.bin:3C66 #3C66

Address Hex
\$+20 14 00 16 00 18 DE 49 77 08 00 0A 00 F8 D0 49 77 ...DIW...DIW
\$+10 00 00 02 00 24 5C 49 77 18 00 1A 00 10 DE 49 77 ...DIW...DIW
\$+00 00 1E 00 1C DE 49 77 02 00 04 00 F0 D0 49 77 ...LDIW...DIW
\$+10 00 04 4E 77 80 77 4E 77 00 00 10 F1 59 77 00W.WW.WW.WW
\$+20 00 D2 4E 77 80 77 4E 77 90 F0 59 77 10 F1 59 77 00W.WW.WW.WW
\$+30 00 00 00 00 57 14 01 E2 46 15 C5 43 AS FE 00 80 ...W..AF.AC.W
\$+40 EE E3 D3 F0 06 00 00 A4 DC 49 77 01 00 00 00 1800...DIW...
\$+50 9A 88 13 35 96 5D 8D 4F 8E 2D A2 44 02 25 F9 3A ...S.WO..CD.WU:
\$+60 06 00 01 00 88 38 49 77 02 00 00 00 F3 38 3F 4A ...DIW..W.W

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused The data has been copied to clipboard. Time Wasted Debugging: 0:00:41:49

Installed service winpmem

Started service winpmem

Memory Info:

CR3: 0x1ae002

NtBuildNumber: 0x6658

KernelBase: 0xfffff8017fa00000

KPCR:

- 0xfffff8010fbf8000

- 0xfffff80107d9d000

NtBuildNumberAddr: 0xfffff8018080a6cc

Run:

- BaseAddress: 0x1000

NumberOfBytes: 0x9f000

- BaseAddress: 0x100000

NumberOfBytes: 0xddda2000

- BaseAddress: 0xddfb6000

NumberOfBytes: 0xb37000

- BaseAddress: 0xdedff000

NumberOfBytes: 0x32e000

- BaseAddress: 0xdf133000

NumberOfBytes: 0x3b000

- BaseAddress: 0x100000000

NumberOfBytes: 0x1cc00000

Setting sparse output file TeslaCrypt_memory.raw

Padding 1 pages from 0x0

Copying 159 pages (0x9f000) from 0x1000

Padding 96 pages from 0xa0000

Copying 908706 pages (0xddda2000) from 0x100000

Padding 276 pages from 0xdddea2000

Copying 2871 pages (0xb37000) from 0xddfb6000

Padding 786 pages from 0xdeaed000

Copying 814 pages (0x32e000) from 0xdedff000

Padding 6 pages from 0xdf12d000

```

Setting sparse output file TeslaCrypt_memory.raw
Padding 1 pages from 0x0
Copying 159 pages (0x9f000) from 0x1000
Padding 96 pages from 0xa0000
Copying 908706 pages (0xdda2000) from 0x100000
Padding 276 pages from 0xddea2000
Copying 2871 pages (0xb37000) from 0xddfb6000
Padding 786 pages from 0xdeaed000
Copying 814 pages (0x32e000) from 0xdedff000
Padding 6 pages from 0xdf12d000
Copying 59 pages (0x3b000) from 0xdf133000
Padding 134802 pages from 0xdf16e000
Copying 117760 pages (0x1cc00000) from 0x100000000
Completed imaging in 1m59.8373691s
Stopped service winpmem
Removing driver from C:\Users\janet\AppData\Local\Temp\979896974.sys
PS C:\Users\janet\Downloads>
PS C:\Users\janet\Downloads> Get-Item .\TeslaCrypt_memory.raw | Select-Object Name,Length,LastWriteTime

```

Window 11 (CIP-A106_Lab03_Clean_Baseline) [Running] - Oracle VirtualBox

File Machine View Input Devices Help

System Informer [DESKTOP-VTGMKK4\janet]

System View Tools Users Help

Refresh Options Find handles or DLLs System information

Search Processes (C)

Processes	Services	Network	Disk	Firewall	Devices		
Name	PID		53... CPU	4.03 kB/s I/O total r...	4.73 GB Private b...	User name	Description
svchost.exe	6260				1.55 MB		Host Process for Windows Ser...
svchost.exe	4424				2.45 MB		Host Process for Windows Ser...
svchost.exe	5832				14.78 MB		Host Process for Windows Ser...
lsass.exe	828				5.75 MB		Local Security Authority Proce...
fontdrvhost.exe	952				1.38 MB		Usermode Font Driver Host
csrss.exe	680	0.09	48 B/s	2.07 MB			Client Server Runtime Process
winlogon.exe	736			2.56 MB			Windows Logon Application
fontdrvhost.exe	960			6.76 MB			Usermode Font Driver Host
dmv.exe	884	0.66		73.38 MB			Desktop Window Manager
explorer.exe	4872	0.27	2.43 kB/s	118.89 MB	DESKTOP-VTGMKK4\jan		Windows Explorer
SecurityHealthSystray.exe	6284			1.77 MB	DESKTOP-VTGMKK4\jan		Windows Security notification...
powershell.exe	2388	0.02		92.67 MB	DESKTOP-VTGMKK4\jan		Windows PowerShell
conhost.exe	7116			1.36 MB	DESKTOP-VTGMKK4\jan		Console Window Host
VBoxTray.exe	7264		28 B/s	2.88 MB	DESKTOP-VTGMKK4\jan		VirtualBox Guest Additions Tra...
OneDrive.exe	4748			27.08 MB	DESKTOP-VTGMKK4\jan		Microsoft OneDrive
OneDrive.Sync.Servi...	4032	0.02		50.14 MB	DESKTOP-VTGMKK4\jan		Microsoft OneDrive Sync Servi...
pestudio.exe	1008	47.85		2.85 GB	DESKTOP-VTGMKK4\jan		Malware Initial Assessment ...
x32dbg.exe	7208	0.23		49.8 MB	DESKTOP-VTGMKK4\jan		x64dbg
demo1_ransomware....	7068			2.1 MB	DESKTOP-VTGMKK4\jan		nah nahApp
powershell.exe	7132	0.02		57.64 MB	DESKTOP-VTGMKK4\jan		Windows PowerShell
conhost.exe	9088			3.25 MB	DESKTOP-VTGMKK4\jan		Console Window Host
MoNotificationUx.exe	6624			2.21 MB	DESKTOP-VTGMKK4\jan		Notifications host process
SystemInformer.exe	5604	1.61		29.43 MB	DESKTOP-VTGMKK4\jan		System Informer

Activate Windows
Go to Settings to activate

CPU usage: 53.86% | Physical memory: 2.14 GB (54.54%) | Free memory: 1.79 GB (45.46%)